Chapter 5 (Day 5)

# Von Neumann vs. Harvard Architecture & The Physical Memory Bus Bottleneck

## 5.0 The Stored-Program Revolution (History & Origins)

To understand modern CPU bottlenecks, we must return to the dawn of electronic computing. The very definition of "software" did not exist in the early 1940s. A computer was not a universal machine; it was a rigid, physical calculator built to solve one specific differential equation.

### The ENIAC & The Physical Program (1945)

The ENIAC (Electronic Numerical Integrator and Computer), completed in 1945 at the University of Pennsylvania, was the world's first general-purpose electronic digital computer. However, "programming" the ENIAC did not involve typing code. It required weeks of physical labor. A team of brilliant female engineers (the "ENIAC Girls," including Jean Bartik and Betty Holberton) had to literally walk inside the 30-ton machine and manually plug hundreds of thick black patch cables into different sockets and flip thousands of mechanical switches to route the electrical logic for a new calculation.

In the ENIAC era, the *program* was the physical hardware wiring. The *data* was inputted via punch cards. Changing the program meant rebuilding the machine.

### John von Neumann & The EDVAC Report

In 1945, the brilliant Hungarian-American mathematician John von Neumann was consulting on the ENIAC's successor, the EDVAC. He synthesized discussions with the ENIAC's inventors, J. Presper Eckert and John Mauchly, and authored a monumental 101-page document titled "*First Draft of a Report on the EDVAC.*"

This document proposed a radical, world-changing abstraction: **The Stored-Program Concept**. Von Neumann argued that the instructions dictating how the machine should behave could be encoded as binary numbers, exactly like the data it was calculating. Therefore, both the *Instructions* and the *Data* could be stored inside the exact same electronic memory array.

> **The Birth of Software (Indigo)**
> The moment instructions were placed into RAM alongside data, the physical wiring of the computer never needed to change again. To run a different application, you simply loaded a different sequence of binary numbers into memory. Hardware and Software were permanently divorced. While Eckert and Mauchly deeply resented von Neumann for taking sole credit for the idea in his report, the resulting architecture was forever named the **Von Neumann Architecture**.

## 5.1 The Pure Von Neumann Architecture

A pure Von Neumann machine is defined by a single, unified physical memory holding both code and data, accessed by a single central processing unit through a shared communication pathway.

### The Tripartite System Bus

The CPU communicates with the main memory via the "System Bus," which is physically composed of three distinct copper wire arrays etched into the motherboard:

1. **The Address Bus:** A unidirectional bus driven by the CPU. If a CPU has a 32-bit address bus, it has 32 physical pins dedicated to outputting $2^{32}$ ($4,294,967,296$) distinct electrical addresses, strictly limiting the system to $4\text{ GB}$ of RAM. A 64-bit address bus can theoretically address $16\text{ Exabytes}$.
2. **The Data Bus:** A bidirectional bus. Once the CPU asserts an address, the RAM drives the requested binary data back to the CPU over these wires, or the CPU drives data to the RAM to be written. The width of this bus (e.g., 64-bit) dictates the memory throughput per clock cycle.
3. **The Control Bus:** Contains operational signals like `Read Enable`, `Write Enable`, and the system clock pulse, ensuring the CPU and RAM interact synchronously.

### The Sequential Execution Trap

Because there is only one physical Data Bus connecting the CPU to the memory, the CPU is subjected to a strict, inescapable sequential execution cycle:

- **Cycle 1 (Fetch):** The CPU asserts the `RIP` (Instruction Pointer) on the Address Bus. The RAM sends the opcode (the instruction) back over the Data Bus.
- **Cycle 2 (Decode & Fetch Operand):** The CPU decodes the opcode. If the instruction says "Add Variable X," the CPU must assert the address of Variable X on the Address Bus. The RAM sends the data over the Data Bus.
- **Cycle 3 (Execute & Write):** The ALU computes the sum. If the result must be saved, the CPU asserts the destination address on the Address Bus and sends the answer over the Data Bus.

> **The Bus Conflict (Orange)**
> Notice the fatal structural limitation: **The CPU cannot fetch the next instruction while it is reading or writing data.** The physical copper wires of the Data Bus are already occupied. The CPU pipeline must brutally stall its Instruction Fetch stage until the Data Memory Access stage completes.

## WHITEBOARD & SYSTEMS WORKSPACE

REF: CH05_VON_NEUMANN_BUS

**The Stored-Program Isomorphism:** In a Von Neumann machine, memory is untyped. A 64-bit block of `1`s and `0`s in RAM could be a floating-point number, a string of ASCII characters, or a compiled `add rax, rbx` assembly instruction. Only the CPU's current execution context dictates how the data is interpreted.

## 5.2 The Von Neumann Bottleneck (The Memory Wall)

In 1977, John Backus (inventor of FORTRAN) won the Turing Award. In his legendary acceptance speech, he coined a phrase that defines modern hardware engineering: **The Von Neumann Bottleneck**.

> "Surely there must be a less primitive way of making big changes in the store than by pushing vast numbers of words back and forth through the Von Neumann bottleneck. Not only is this tube a literal bottleneck for the data traffic of a problem, but, more importantly, it is an intellectual bottleneck that has kept us tied to word-at-a-time thinking." — John Backus

### The Latency Divergence

In the 1980s, CPU speeds and RAM speeds were relatively identical. However, Dennard Scaling allowed CPU frequencies to double every two years, rocketing to $5.0\text{ GHz}$ (a $0.2\text{ ns}$ cycle time). Meanwhile, Dynamic RAM (DRAM) relies on microscopic capacitors that must physically drain and fill with electrons. DRAM physics scaled much slower.

Today, a modern DDR5 RAM access takes approximately $70\text{ to }100\text{ nanoseconds}$.

- **CPU Clock Cycle:** $0.2\text{ ns}$
- **RAM Access Latency:** $80.0\text{ ns}$
- **The Gap:** A single main memory fetch costs the CPU **400 clock cycles**.

If a CPU executing at 5 GHz had to reach across the Von Neumann bus to main RAM for every instruction and data fetch, it would spend 99.7% of its life completely frozen, waiting for electrons to travel across the motherboard. The processor would effectively operate at $12\text{ MHz}$. The shared bus became the ultimate choke point in computing history.

### Data Starvation

Modern super-scalar ALUs can execute 4 or 5 instructions simultaneously per clock cycle. But if the Von Neumann bus can only deliver one word of data at a time, the ALUs starve. The CPU becomes a high-performance Ferrari stuck permanently in a one-lane traffic jam.

## 5.3 The Harvard Architecture

While von Neumann was designing the EDVAC, Howard Aiken at Harvard University was designing the Harvard Mark I (1944). Unlike the EDVAC, Aiken's machine utilized a fundamentally different layout.

### The Topology of Separation

The **Harvard Architecture** physically separates Instructions and Data into two entirely isolated memory modules, each with its own dedicated Address and Data Bus.

- **Instruction Memory:** Has its own copper pathways to the CPU's Fetch unit.
- **Data Memory:** Has its own copper pathways to the CPU's Memory Read/Write execution units.

### The Throughput Miracle

Because the buses are physically isolated, there is no contention. In a 5-stage pipeline, the CPU can be fetching Instruction N from the Instruction RAM over Bus A, while simultaneously writing the result of Instruction N-2 to the Data RAM over Bus B during the exact same clock cycle. Theoretical memory bandwidth is instantly doubled.

### The Wasted Space Dilemma

If pure Harvard architecture is so vastly superior for throughput, why don't modern desktop computers use it for main RAM? **Resource Fragmentation.**

If a motherboard has $16\text{ GB}$ of RAM, a pure Harvard design requires permanently soldering $8\text{ GB}$ for Instructions and $8\text{ GB}$ for Data. If a video game requires $12\text{ GB}$ of graphics data but only $2\text{ GB}$ of code, the application will crash with an Out-of-Memory exception, even though there are $6\text{ GB}$ of empty space sitting uselessly on the Instruction side of the board. General-purpose computing requires dynamic, flexible memory boundaries.

> **Microcontrollers: The Harvard Survivors (Emerald)**
> Pure Harvard architecture completely dominates the embedded microcontroller market (Arduino, AVR, PIC chips). An embedded chip runs a single, compiled C-program forever. The program code is burned permanently into Flash ROM (Instruction Memory), and the runtime variables exist in a tiny SRAM (Data Memory). Because the code size is known and fixed at compile time, fragmentation is irrelevant, and the microcontroller enjoys perfect 1-cycle deterministic execution predictability.

## WHITEBOARD & SYSTEMS WORKSPACE



**Instruction Width Independence:** In a Harvard architecture, the Instruction Bus and Data Bus do not have to be the same size. A microcontroller can have a 14-bit wide Instruction Flash ROM to store optimized opcodes, while featuring an 8-bit wide Data SRAM for standard byte manipulation. This is impossible in Von Neumann.

## 5.4 The Super-Harvard Architecture (SHARC)

Before exploring the modern desktop compromise, we must examine extreme high-performance variations. Digital Signal Processors (DSPs) are specialized chips used in audio processing, radar, and software-defined radio. They must process endless streams of data (like applying a Fast Fourier Transform to a $192\text{ kHz}$ audio stream) with zero latency tolerance.

DSPs utilize the **Super-Harvard Architecture (SHARC)**. It extends the Harvard design by adding:

- **Dual Data Memories:** Not just an Instruction and Data split, but *two* independent Data Memories (Data X and Data Y).
- **Instruction Cache:** A tiny, hyper-fast cache sitting in front of the Instruction Memory.

### The MAC Operation (Multiply-Accumulate)

The core of signal processing is the FIR (Finite Impulse Response) filter, which relies entirely on the mathematical operation: $A = A + (X \times Y)$.

In a pure Von Neumann machine, fetching $X$, fetching $Y$, multiplying, adding, and saving takes over a dozen clock cycles. In a SHARC DSP, the CPU can:

1. Fetch the next MAC instruction from the Instruction Cache.
2. Fetch the $X$ audio sample from Data Memory 1 over Bus 1.
3. Fetch the $Y$ filter coefficient from Data Memory 2 over Bus 2.
4. Multiply and accumulate the result in the hardware MAC unit.

**All of this occurs in a single physical clock cycle.** This architectural specialization allows a $500\text{ MHz}$ DSP to absolutely destroy a $4.0\text{ GHz}$ Von Neumann CPU in raw audio signal processing throughput.

## 5.5 The Modern Compromise: Modified Harvard Architecture

Modern general-purpose CPUs (Intel Core, AMD Ryzen, Apple M-Series) cannot use pure Harvard (due to RAM fragmentation) and cannot use pure Von Neumann (due to the 400-cycle bus bottleneck). The solution is a masterpiece of architectural deception: **The Modified Harvard Architecture**.

### Macro-Level: Von Neumann

Outside the CPU die, the motherboard operates as a pure Von Neumann machine. You buy unified sticks of DDR5 RAM (e.g., $32\text{ GB}$). The operating system dynamically allocates chunks of this RAM for Chrome's executable code and Chrome's tab data seamlessly. There is only one external Memory Controller and one external physical system bus.

### Micro-Level: Harvard

Inside the CPU die, the architecture radically changes. To feed the superscalar pipeline, the CPU relies on microscopic, sub-nanosecond SRAM caches. The **Level 1 (L1) Cache** is strictly, physically split into two separate domains:

- **L1i (Instruction Cache):** Typically $32\text{ KB}$. Wired directly to the pipeline's Instruction Fetch (IF) unit.
- **L1d (Data Cache):** Typically $32\text{ KB}$ to $48\text{ KB}$. Wired directly to the pipeline's Memory Access (MEM) execution unit.

Because the CPU executes out of the L1 cache 95% of the time, the execution pipeline experiences perfect, contention-free Harvard parallel bandwidth. It fetches opcodes and data simultaneously without bus collisions.

> **The L2/L3 Cache Dilemma (Pink)**
>
> If splitting L1 into Instruction and Data is so powerful, why are L2 and L3 caches universally unified (Von Neumann)?
>
> **Answer:** Statistical utilization limits. Instruction code footprints are usually small and highly localized (a few loops), while data footprints are massive and unpredictable (large arrays). If L2 was split $1\text{MB}/1\text{MB}$, the Data cache would constantly overflow while the Instruction cache sat 90% empty. A unified L2 dynamically adapts to the workload, storing 10% instructions and 90% data if required, optimizing expensive silicon real estate while relying on the L1 split to handle the immediate pipeline throughput.

## WHITEBOARD & SYSTEMS WORKSPACE

REF: CH05_MODIFIED_HARVARD_CACHE

## 5.6 Security Ramifications: The Curse of Von Neumann

The Von Neumann architecture's greatest strength—that code and data share the exact same memory—is also the greatest security vulnerability in the history of computer science. If the CPU cannot physically distinguish between an integer variable and an executable instruction, it can be tricked into executing user data.

### Smashing the Stack for Fun and Profit (1996)

In 1996, security researcher Elias Levy (Aleph One) published a paper detailing the canonical **Stack-Based Buffer Overflow**.

When a C function is called, the CPU pushes the **Return Address** (the `RIP` pointer telling the CPU where to go when the function finishes) onto the Call Stack. Right below this return address, the program allocates space for local variables (like a 64-byte `char` array for a username).

If the programmer uses an unsafe function like `strcpy()`, which copies user input into the array without checking the length, a hacker can input a 100-byte string. The first 64 bytes fill the array. The remaining 36 bytes physically overflow the array boundary, continuing up the stack memory, and **overwrite the saved Return Address**.

### The Malicious Payload (Shellcode)

The hacker doesn't just overwrite the return address with garbage. They overwrite it with the exact memory address pointing *back into the array they just overflowed*. Inside that array, they have hidden raw binary assembly opcodes (Shellcode) disguised as string data.

When the C function finishes, it executes the `ret` instruction. The CPU pops the corrupted Return Address off the stack and jumps to it. Because it is a Von Neumann machine, the CPU blindly assumes the string data in the array is now executable code, and begins executing the hacker's payload, granting them full control of the server.

### Hardware Defenses: Enforcing Harvard on Von Neumann

To stop buffer overflows, AMD and Intel realized they had to introduce Harvard-style separation logically, if not physically. They introduced a new feature to the MMU (Memory Management Unit) page tables: **The NX Bit (No-Execute)**, also known as DEP (Data Execution Prevention).

With the NX bit, the operating system can mark regions of RAM with a **W^X (Write XOR Execute)** policy:

- **Data regions (Heap, Stack):** Marked as Writable but NOT Executable.
- **Code regions (Text segment):** Marked as Executable but NOT Writable.

Now, if a hacker overflows the stack and points the `RIP` to their shellcode string, the CPU's MMU sees that the Stack page has the NX bit set. The CPU instantly halts, throwing a hardware `Segmentation Fault` (Access Violation), refusing to execute the data.

> **The Return-Oriented Programming (ROP) Countermeasure**
>
> Hackers evolved. If they can no longer execute their own injected code due to the NX bit, they exploit the code that is *already* marked as executable in the program. By carefully overwriting the stack with a massive chain of fake return addresses, they force the CPU to bounce around the existing executable memory, executing tiny fragments of existing code ending in a `ret` instruction (called *Gadgets*). By chaining thousands of gadgets together (a ROP Chain), they synthesize a malicious payload using the host program's own legitimate assembly instructions, perfectly bypassing DEP and the NX bit.

## WHITEBOARD & SYSTEMS WORKSPACE



**The JIT Security Paradox:** Just-In-Time compilers (like the V8 JavaScript engine or the .NET CLR) fundamentally violate W^X policies. Because they generate machine code on the fly, they require memory pages that are simultaneously Writable (to emit the JITed assembly) and Executable (to run it). This makes JIT buffers a massive target for browser exploit chains.

## 5.7 Software Relevancy: Mechanical Sympathy

Understanding the L1i and L1d split in the Modified Harvard architecture directly influences how elite software engineers organize their code and memory layouts. Ignorance of this split results in invisible, catastrophic performance drops.

### L1 Data Cache Sympathy (L1d Thrashing)

The L1d cache relies heavily on spatial locality. When you request a 4-byte integer from RAM, the CPU actually fetches an entire 64-byte Cache Line.

- **Array of Structs (AoS):** If you have an array of `Player` objects, and you run a loop to sum up everyone's `Health`, you fetch the Player object, read the 4-byte health, and skip over the 60 bytes of Inventory, Name, and Position data. You just wasted 90% of your L1d cache bandwidth pulling useless data over the bus.
- **Struct of Arrays (SoA):** Data-Oriented Design (DOD) reorganizes this. You create a single array containing only `Health` integers. Now, one 64-byte cache line pull delivers 16 sequential Health values. The ALU processes them instantly, achieving 100% L1d cache utilization and maximizing bus throughput.

### L1 Instruction Cache Sympathy (L1i Thrashing)

The L1i cache is incredibly small ($32\text{ KB}$). If your code executing inside a tight loop exceeds $32\text{ KB}$ of compiled assembly instructions, the end of the loop will overwrite the instructions for the beginning of the loop in the cache. When the loop restarts, the CPU suffers an L1i cache miss and must stall the pipeline to re-fetch the start of the loop from L2 cache. This is known as **Instruction Thrashing**.

How does a developer accidentally bloat a tight loop beyond the $32\text{ KB}$ L1i limit?

1. **Aggressive Loop Unrolling:** Compilers sometimes copy-paste the inside of a loop 10 times in a row to avoid the branch condition check (`i++ < N`). If the loop body is large, this copy-pasting blows out the L1i footprint.
2. **Excessive Function Inlining:** Marking every helper function as `[MethodImpl(MethodImplOptions.AggressiveInlining)]` forces the compiler to inject the function's raw assembly directly into the call site. While it saves the 5-cycle function call overhead, repeating this 50 times across a hot path destroys the L1i cache, making the program radically slower overall.
3. **Deep Object-Oriented Virtual Dispatch:** Heavy polymorphism and deep inheritance trees require the CPU to navigate massive v-tables (Virtual Method Tables). The code for these disparate inherited methods is scattered randomly across memory, making it impossible for the L1i prefetcher to load the code sequentially, resulting in constant pipeline stalls.

> **The Production Profiling Benchmark (Emerald)**
> When tuning high-frequency trading engines, engineers don't just look at total execution time. They run Linux `perf stat` to analyze hardware PMU (Performance Monitoring Unit) counters. If they see `L1-icache-load-misses` spiking, they immediately restrict function inlining and compact their control flow. Optimizing for instruction cache density is often more impactful than optimizing the algorithmic Big-O complexity in hot paths.

## WHITEBOARD & SYSTEMS WORKSPACE



**The Code Density Fallacy:** Writing "less code" in a high-level language does not guarantee L1i cache density. A single 1-line LINQ query in C# can dynamically generate hundreds of lines of invisible closure state-machine allocations and generic specialization assembly, silently blowing out the instruction cache.

## 5.8 Chapter Verification, Checklist & Quiz

### Verification Checklist

I can explain the Stored-Program concept and why it divorced hardware wiring from software logic.
I can define the three components of the System Bus (Address, Data, Control).
I understand the Von Neumann Bottleneck and why RAM latency starves modern CPUs.
I can describe the physical layout of a pure Harvard Architecture and its throughput advantages.
I understand the memory fragmentation flaw that prevents pure Harvard use in general computing.
I can identify why microcontrollers (Arduino/AVR) utilize pure Harvard architectures.
I can explain the Modified Harvard Architecture (Unified Main RAM + Split L1i/L1d caches).
I understand how a Buffer Overflow leverages the Von Neumann unified memory to execute injected data.
I can explain how the NX Bit (W^X) artificially enforces Harvard isolation for security.
I understand how excessive function inlining destroys performance via L1 Instruction Cache thrashing.

## Comprehensive Examination

1. What is the fundamental, defining characteristic of a pure Von Neumann machine?

A) The CPU utilizes separate buses for Address and Data mapping.
B) The system relies on a multi-stage execution pipeline to increase throughput.
C) Instructions (code) and Data (variables) are stored in the exact same physical memory space and share a single bus array.
D) The architecture uses dynamic RAM (DRAM) instead of static RAM (SRAM) for main memory.

2. According to John Backus, what is the primary physical cause of the "Von Neumann Bottleneck"?

A) The CPU generates too much heat when evaluating instructions sequentially.
B) The single shared Data Bus forces the CPU to stall instruction fetches while reading or writing data, exacerbated by the massive latency gap between fast CPU clocks and slow DRAM response times.
C) The Address bus is physically restricted to 32 bits, capping memory at 4 GB.
D) The lack of an L1 cache in early processor topologies.

3. Why is the pure Harvard Architecture unsuitable for a modern desktop operating system running general-purpose applications?

A) It does not support floating-point mathematics inherently.
B) It restricts the use of virtual memory and translation lookaside buffers (TLBs).
C) It requires physically partitioned RAM for code and data, leading to catastrophic resource fragmentation and out-of-memory errors for unpredictable workloads.
D) The dual buses create severe electromagnetic interference (EMI) at gigahertz speeds.

## WHITEBOARD & SYSTEMS WORKSPACE

REF: CH05_QUIZ_SCRATCHPAD_1

4. Which of the following accurately describes the "Modified Harvard Architecture" utilized by all modern Intel, AMD, and Apple processors?

A) Main memory (DRAM) is unified, but the internal L1 cache is strictly split into dedicated Instruction (L1i) and Data (L1d) caches to feed the pipeline concurrently.
B) The CPU uses Harvard for main memory but unifies the L3 cache.
C) The architecture uses software microcode to simulate a dual-bus structure over a single physical copper trace.
D) Main memory is strictly split, but the CPU pipeline executes instructions out of order to simulate a unified space.

5. How does the No-Execute (NX) bit hardware feature defend against classic Stack-Based Buffer Overflow attacks?

A) It encrypts the return address (`RIP`) pointer on the stack using a random canary value.
B) It enforces a W^X (Write XOR Execute) policy, preventing the CPU from executing assembly instructions that reside in memory pages marked for variable data (like the Stack or Heap).
C) It physically disconnects the data bus if a string exceeds its allocated integer array boundary.
D) It randomizes the memory locations of shared libraries (ASLR).

6. An engineer marks 50 utility functions with aggressive compiler inline attributes. Benchmarks show the code is now 20% slower. What architectural threshold was likely violated?

A) The CPU experienced Data Hazard RAW stalls due to register depletion.
B) The inlined assembly footprint exceeded the 32 KB capacity of the L1 Instruction Cache (L1i), causing severe instruction thrashing and pipeline starvation.
C) The expanded code caused a stack overflow, expanding into the Heap.
D) The L2 cache failed to synchronize via the MESI protocol.

7. In the Super-Harvard Architecture (SHARC) used by Digital Signal Processors, why are there two independent Data Memories in addition to the Instruction Cache?

A) To separate incoming network packets from outgoing packets.
B) To allow the Multiply-Accumulate (MAC) unit to fetch two separate operands (e.g., an audio sample and a filter coefficient) simultaneously in a single clock cycle.
C) To provide a backup failover memory in high-radiation aerospace environments.
D) To partition integer variables from floating-point variables.

8. What technique do modern hackers use to bypass the NX Bit / DEP security mechanisms on Von Neumann machines?

A) Cross-Site Scripting (XSS).
B) Return-Oriented Programming (ROP) Chains, which string together existing executable code snippets instead of injecting new executable data.
C) SQL Injection to alter the MMU page tables directly.
D) Thermal fault injection via aggressive overclocking.

## Systems Writing Prompts

### Prompt 1: The Bottleneck Equation

Explain the mechanics of the Von Neumann Bottleneck. Contrast the physical latency of a modern CPU clock cycle against the access latency of DDR5 RAM. Detail why adding more ALUs to a processor does not increase performance for memory-bound workloads.

### Prompt 2: The Security Paradox

The Stored-Program concept treats code and data as identical binary strings. Explain how this brilliant architectural abstraction directly enables Buffer Overflows. Describe the physical stack layout (Local Variables vs. Return Address) and the mechanical execution sequence of a malicious payload.

## WHITEBOARD & SYSTEMS WORKSPACE

REF: CH05_PROMPT_SCRATCHPAD

### CHAPTER 5 TECHNICAL ANSWER KEY

**1. (C)** The defining trait of Von Neumann is the unified memory model. The lack of distinction between data and executable code revolutionized computing flexibility but introduced severe bus contention bottlenecks and security vulnerabilities.

**2. (B)** The CPU is orders of magnitude faster than DRAM. Because it must use the same physical bus to fetch instructions and read/write data, it cannot do both concurrently. The pipeline starves while waiting for the RAM to respond.

**3. (C)** Pure Harvard requires hard-wired partitions (e.g., exactly 8GB for code, 8GB for data). If an application's memory profile dynamically shifts and requires 10GB of data, the system crashes from fragmentation, leaving empty instruction memory stranded and unusable.

**4. (A)** To achieve Harvard-level parallel pipeline throughput without suffering Von Neumann fragmentation, modern chips use a Unified DDR RAM but strict physically separated L1i and L1d SRAM caches on the die.

**5. (B)** By explicitly segregating memory pages via the MMU (W^X), the OS guarantees that any area the user can write to (data buffers) is chemically stripped of its execute privileges, instantly faulting if the `RIP` is pointed there.

**6. (B)** Inlining removes function call overhead but physically copies the assembly payload into every call site. This massive code bloat evicts other critical instructions from the tiny 32KB L1i cache. When the loop restarts, the CPU must stall to re-fetch the instructions from L2, destroying pipeline throughput.

**7. (B)** DSP algorithms like FIR filters require a Multiply-Accumulate (MAC) operation involving two distinct data inputs simultaneously. Dual data memories and buses allow both inputs to be fetched concurrently in 1 clock cycle without stalling the pipeline.

**8. (B)** Since hackers cannot execute their injected stack payloads due to the NX bit, they pivot to ROP. By chaining together tiny, preexisting fragments of application code that end in `ret`, they manipulate the stack pointers to achieve arbitrary code execution via the application's own legitimate, executable memory.

**Prompt 1:** A modern CPU cycles every $0.2\text{ns}$, while DDR5 latency is ~$80\text{ns}$ (a 400-cycle gap). The Von Neumann bottleneck is the inability to fetch instructions and data simultaneously over the shared bus. If a workload is memory-bound (e.g., massive array traversals), the execution units sit idle waiting for data. Adding 10 more ALUs provides zero benefit because the physical copper data pipeline is saturated, stalling the entire architecture.

**Prompt 2:** Because RAM is untyped binary, user inputs and kernel code look identical to the CPU. If `strcpy` overwrites an array boundary, it bleeds into the call stack, overwriting the saved Return Address. By pointing this corrupted Return Address to the injected shellcode inside the array, the CPU will jump there upon function exit and execute the text input as if it were a valid compiled program.

## WHITEBOARD & SYNTHESIS WORKSPACE

REF: CH05_FINAL_SYNTHESIS